Porting the RTOS to Cortex-M4 (STM32F4)

RTOS Project

2026-10-19

Contents

1. Context and Scope **1**

2. Toolchain and Build Configuration **2**
- 2.1 CPU flag . . . 2
- 2.2 FPU flags — a decision, not a default . . . 2
- 2.3 New BOARD branch . . . 3

3. FPU Context Switching (deferred — read if/when enabling hardware float) **3**

4. New BSP: `bsp/stm32f407/` **4**
- 4.1 `clock.c` — clock tree bring-up . . . 4
- 4.2 `systick.c` — expected to port unchanged . . . 4

5. New Linker Script: `linker/stm32f407.ld` **4**

6. Debug and Emulation Tooling **5**
- 6.1 Renode platform: `scripts/renode/stm32f407.repl` (+ `_run.resc`/`_debug.resc`) . . . 5
- 6.2 OpenOCD RTT config: `scripts/openocd/stm32f407_rtt.cfg` . . . 5

7. Startup Code — Vector Table (informational, likely no action needed) **6**

8. Validation Plan **6**

9. Summary Table **7**

1. Context and Scope

This RTOS currently targets a single Cortex-M3 part: the STM32F103 (“Blue Pill”), built through `arch/arm_cm/` (the shared ARM Cortex-M port) and `bsp/stm32f103/` (the chip-specific board support package). This document is a step-by-step plan for adding a second target: an STM32F4-family Cortex-M4 part (e.g. STM32F407).

Why this is mostly additive, not a rewrite. Cortex-M3 and Cortex-M4 share the same base architecture (ARMv7-M), the same exception model, the same MPU architecture (PMSAv7), and the same core debug peripherals (SysTick, DWT, NVIC, SCB) at the same addresses. The M4 adds: a single- precision FPU (optional per-part), a handful of DSP/SIMD instructions (`SMMUL`, `SSAT`,

etc. — not used by a C-only kernel), and minor exception-model extensions that don't affect this codebase. Concretely, that means:

- `arch/arm_cm/port.c` (PendSV context switch, SVC dispatch, MPU setup, IRQ save/restore) needs **no change** unless hardware floating point is enabled (see §3).
- `arch/arm_cm/startup.c` (vector table, `Reset_Handler`) needs **no change** — the core exception vector table (indices 0–15) is identical between M3 and M4.
- `bsp/stm32f103/systick.c` needs **no change in logic** — SysTick is an ARMv7-M core peripheral, not a vendor peripheral, and already has a comment to this effect.

Everything that *does* need to change is either (a) build configuration, (b) code that pokes vendor-specific registers (RCC/PWR/FLASH), or (c) the memory map (linker script), because those are the parts that are genuinely different between the STM32F1 and STM32F4 silicon families — not between M3 and M4 as architectures.

2. Toolchain and Build Configuration

File: `Makefile`

2.1 CPU flag

Change (or branch) the compiler target:

```
CPU := cortex-m4
```

Today this is a single unconditional `CPU := cortex-m3` (Makefile line 10). Since the repo will support two chip families, make this conditional on `BOARD` the same way `LD_SCRIPT`/`BSP_SRC` already are (Makefile lines 22–34), so `cortex-m3` is still used for the F103 boards and `cortex-m4` only for the new F4 board.

Why: `-mcpu` controls instruction scheduling and selection (`arm-none-eabi-gcc` will emit M4-appropriate code, e.g. it becomes legal/efficient to use `SDIV`/`UDIV`, which M3 also has, and DSP extensions if you ever opt into them). It does not by itself change calling convention or ABI, so no other code changes are forced by this flag alone.

2.2 FPU flags — a decision, not a default

Grep of the current tree shows **zero** uses of `float`/`double` anywhere in `kernel/`, `include/`, `app/`, or `arch/`. That means there is no forcing requirement to touch the FPU at all for this port.

Recommendation: do not add `-mfpu`/`-mfloat-abi` flags yet. Build the M4 target the same way as M3 — soft float, no VFP instructions emitted, no change to the exception frame. This keeps `PendSV_Handler` and `SVC_Handler` byte-for-byte identical between the two targets and minimizes the risk surface of this port.

If/when a task needs floating point, revisit with:

```
CFLAGS += -mfpu=fpv4-sp-d16 -mfloat-abi=softfp   # or -mfloat-abi=hard
```

and implement the kernel change described in §3. Flagging this now so it isn't forgotten and silently miscompiled later (mixing float ABIs between object files is a classic, hard-to-diagnose Cortex-M bug).

2.3 New BOARD branch

Add a new branch alongside the existing `stm32f103`/`stm32f103c8t6` ones (Makefile lines 22–34):

```
else ifeq ($(BOARD),stm32f407)
CPU                   := cortex-m4
LD_SCRIPT             := linker/stm32f407.ld
BSP_SRC               := bsp/stm32f407/systick.c bsp/stm32f407/clock.c
RENODE_RUN_SCRIPT     := scripts/renode/stm32f407_run.resc
RENODE_DEBUG_SCRIPT   := scripts/renode/stm32f407_debug.resc
```

Why a new BOARD rather than overloading the old one: the Makefile's `check-board` logic (lines 88–95) already rebuilds from clean whenever `BOARD`/`APP_SRC`/`HW` changes, specifically so stale objects built for one chip's memory map never get linked against another chip's linker script. Reusing that mechanism is the path of least resistance and keeps `BOARD=stm32f103c8t6` fully working with zero regression risk.

3. FPU Context Switching (deferred — read if/when enabling hardware float)

This section only applies if §2.2's recommendation is later reversed.

Why this is the one real kernel change M4 can force: the M4's FPU adds registers (`S0`–`S31`, `FPSCR`) that live outside the base integer register set saved by `PendSV_Handler` today (`arch/arm_cm/port.c` lines 189–222, which only stacks `r4`–`r11` manually and relies on hardware auto-stacking for `r0`–`r3`/`r12`/`lr`/`pc`/`xpsr`). If a task uses the FPU and the kernel doesn't account for it, a context switch will silently corrupt that task's floating-point state the next time it runs.

If enabling:

1. Set `SCB->CPACR` (`0xE000ED88`) bits `[23:20]` to `0b1111` during boot (`Reset_Handler` or `os_boot_main`) to enable the FPU coprocessor — it's off by reset default, and any VFP instruction before this traps as `UsageFault`.
2. In `PendSV_Handler`, test `CONTROL.FPCA` (bit 2) after the existing `tst lr, #4` check — `FPCA` is set by hardware whenever the interrupted code used the FPU. If set, additionally `vstmdb r0!, {s16-s31}` before `stmdb r0!, {r4-r11}` on save, and the mirrored `vldmia` on restore. (`s0-s15` and `FPSCR` are already covered by the hardware's own automatic stacking when `FPCA` is set — only the "callee-saved" FPU registers `s16-s31` need manual handling, symmetric to how `r4-r11` are handled for the integer side.)
3. Alternatively, adopt ARM's **lazy FPU stacking** (`FPCCR.LSPEN`, default-enabled) to avoid the save/restore cost on tasks that never touch the FPU — more throughput-efficient but adds another status register to reason about when debugging; a naive eager save as in step 2 is simpler to get correct first.
4. `os_exc_frame_t` (port.c lines 54–63) does **not** need extension — it models the integer-only hardware frame, which is unaffected by FPU use; only the manually-saved region changes.

4. New BSP: bsp/stm32f407/

Why a new directory instead of extending bsp/stm32f103/**:** the existing BSP's own header comment says it exists "purely so STM32F103 has its own place to grow chip-specific peripheral code" — the pattern is one BSP directory per chip family, matching how BOARD already selects BSP_SRC per chip in the Makefile. STM32F1 and STM32F4 have different RCC/PWR/FLASH register layouts entirely (different bit positions, different PLL topology), so there is no shared vendor code to factor out yet.

4.1 clock.c — clock tree bring-up

Why this needs a full rewrite, not a tweak: bsp/stm32f103/clock.c implements the F1 clock tree, which is architecturally simple — one HSE crystal, one PLL with a single integer multiplier (×9), feeding SYSCLK directly, with a 2-wait-state Flash setting for 72MHz. The F4 clock tree is a different shape:

- The PLL has **four** programmable dividers (PLLM, PLLN, PLLP, PLLQ) instead of one multiplier — e.g. for an 8MHz HSE driving 168MHz SYSCLK: PLLM=8 (-> 2MHz VCO input), PLLN=336 (-> 672MHz VCO), PLLP=2 (-> 336MHz… adjust per exact target clock/part), PLLQ=7 (for 48MHz USB OTG FS clock, if used).
- F4 requires PWR->CR voltage-scaling (VOS bits) to be set to full scale 1 for max frequency *before* enabling the PLL — the F1 has no equivalent step.
- FLASH->ACR wait states follow a different table on F4 (depends on both voltage scaling and frequency — e.g. 5 wait states at 168MHz under VOS scale 1) and F4 additionally benefits from enabling the ART accelerator (prefetch + instruction/data cache bits in the same register) rather than just the F1's single prefetch-buffer-enable bit.
- APB1/APB2 prescalers have different max frequencies on F4 (42MHz/84MHz typical ceilings) than F1's single APB1 36MHz ceiling, so RCC_CFGR divider selection needs re-deriving, not copying.

Structure this file the same way as the existing one: HSE-on-ready-poll, configure PLL dividers, switch SYSCLK source, poll ready — same pattern, different register map and derivation. Keep the same #ifdef-free style (only compiled in when HW=1), matching the existing clock_init() convention and comment explaining when it's safe to skip (see §6 on Renode).

4.2 systick.c — expected to port unchanged

Confirmed by reading bsp/stm32f103/systick.c: it only touches SYST_CSR/SYST_RVR/SYST_CVR at their fixed ARMv7-M core addresses (0xE000E010–0xE000E018), with no RCC/vendor dependency. Copy the file as-is into bsp/stm32f407/systick.c; the only thing that changes is the cpu_hz value passed in by kernel/main.c/the clock config (OS_CPU_HZ), which reflects the new SYSCLK, not this file's logic.

5. New Linker Script: linker/stm32f407.ld

Why a new script, and why the memory split matters: both existing scripts (linker/stm32f103c8t6.ld, linker/stm32f103.ld) carry an explicit constraint, called out in their own header comments: RAM must split into **two power-of-two halves** (user/kernel), because os_port_mpu_region_configure()

in `arch/arm_cm/port.c` (lines 65–80) encodes MPU region sizes as `log2(size)`, which only works for power-of-two region sizes. `stm32f103c8t6.ld` documents choosing an uneven 16K/4K split specifically because 20K total doesn't divide evenly — the same reasoning applies to whatever RAM size the target F407 part has, and the chosen split must be re-derived, not assumed to be an even 50/50 split.

Concretely, for a part like the **STM32F407VG** (1MB flash / 128KB SRAM, excluding the separate non-DMA-capable 64KB CCM RAM):

```
MEMORY {
  FLASH_USER   (rx)  : ORIGIN = 0x08000000, LENGTH = 512K
  FLASH_KERNEL (rx)  : ORIGIN = 0x08080000, LENGTH = 512K
  RAM_USER     (rwx) : ORIGIN = 0x20000000, LENGTH = 96K
  RAM_KERNEL   (rwx) : ORIGIN = 0x20018000, LENGTH = 32K
}
```

(128K splits evenly as a power of two, so a clean user/kernel ratio can be chosen freely here — unlike the Blue Pill's 20K, pick based on measured usage the way the existing comment describes: "Measured actual usage … ~8.5K user … ~2.1K kernel.") Re-measure for the actual app/kernel footprint on this target rather than copying the ratio above verbatim.

Also decide **whether to use CCM RAM at all**: it's not addressable by any DMA controller on F4, so stacks/buffers placed there would silently break any future DMA-based driver. Simplest and safest for this port: leave it unused for now, matching the "defer it" posture of §2.2/§3.

Otherwise, copy the section structure (`.isr_vector`, `.text.user`, `.text.kernel`, `.data.user`/`.kernel`, `.bss.user`/`.kernel`) verbatim from `stm32f103c8t6.ld` — none of that is chip-specific, only the `MEMORY` block and resulting symbol addresses are.

6. Debug and Emulation Tooling

6.1 Renode platform: `scripts/renode/stm32f407.repl` (+ `_run.resc`/`_debug.resc`)

Why this is needed and why `HW` handling may actually simplify: the existing `stm32f103.repl`/`stm32f103c8t6.r` work because Renode ships or models those platforms, but the F1 RCC block isn't modeled (per the comment in `bsp/stm32f103/clock.c`: "Renode's platform files … don't model RCC at all"), which is *why* the Makefile forces `HW=0` for the Renode-only board today (Makefile lines 44–47). Renode's STM32F4 platform support is more mature and commonly does model RCC — if true for the specific F407 variant chosen, `HW=1` may be usable even under Renode, meaning the emulator and real hardware could run the *exact same* clock bring-up path for the first time in this repo. This needs checking against Renode's actual peripheral model for the chosen part before assuming it, since it changes whether a `stm32f407`-Renode-only board variant (parallel to the existing `stm32f103` vs `stm32f103c8t6` split) is needed at all.

6.2 OpenOCD RTT config: `scripts/openocd/stm32f407_rtt.cfg`

Copy `scripts/openocd/stm32f103c8t6_rtt.cfg` and swap the target config from the F1 target script to `target/stm32f4x.cfg` (bundled with OpenOCD); the SWD adapter config and RTT control-block polling setup are adapter-specific, not chip-specific, and should be unchanged.

7. Startup Code — Vector Table (informational, likely no action needed)

Why this is called out despite needing no change today: `arch/arm_cm/startup.c` (lines 34–49) defines the vector table only through `SysTick_Handler` — no peripheral IRQs are wired up yet for any board. The *core* exception set (`Reset`, `NMI`, `HardFault`, `MemManage`, `BusFault`, `UsageFault`, `SVC`, `DebugMon`, `PendSV`, `SysTick`) is architecturally identical between M3 and M4, so this file needs **zero changes** for this port as it stands.

This will need revisiting the first time an interrupt-driven peripheral driver is added for the F4 (e.g. a UART RX IRQ), because the NVIC vector list *after* `SysTick_Handler` is chip-specific — F4's peripheral IRQ numbering differs from F1's. Not required for this port; noted so it isn't mistaken for an oversight later.

8. Validation Plan

Once the above is wired up, validate in this order — each step assumes the previous one is working, so stop and debug at the first failure rather than proceeding:

1. **Boot and halt-point check.** Build with `BOARD=stm32f407`, flash, confirm `Reset_Handler` runs `os_copy_words`/`os_zero_words` correctly (check `.data` initialized, `.bss` zeroed via `arm-none-eabi-gdb` or RTT print) before `os_boot_main()` is entered.
2. **Clock verification.** Confirm actual SYSCLK matches the configured target — e.g. toggle a GPIO in a tight loop and measure frequency on a scope/logic analyzer, or read back `RCC_CFGR.SWS` to confirm PLL is actually the active source (mirroring the F1 `clock_init`'s own ready-polling pattern).
3. **SysTick/tick subsystem.** Confirm `kernel/tick.c` ticks at the expected rate given the new `OS_CPU_HZ` — wrong `OS_CPU_HZ` relative to actual SYSCLK silently produces a wrong-but-plausible tick rate that's easy to miss without this explicit check.
4. **Context switch.** Run the existing scheduler with ≥2 tasks and confirm `PendSV_Handler` switches correctly — this exercises exactly the code path that would break first if an FPU-context bug were present (it shouldn't be, per §2.2/§3, but this is the test that would catch it).
5. **SVC syscalls.** Exercise each case in `os_port_svc_dispatch` (`arch/arm_cm/port.c` lines 82–127): yield, delay, semaphore take/give, mutex lock/unlock, task create.
6. **MPU enforcement.** Confirm user-mode code faults (`MemManage_Handler`) when attempting to access the kernel RAM/flash region — proves the new linker script's region boundaries and `os_port_mpu_init()`'s size encoding agree for the new memory map.
7. **Priority inheritance regression.** Run `app/pi_test.c` — this is the existing scenario test for the mutex priority-inheritance logic and should behave identically on M4 since none of that logic is architecture-dependent.
8. **RTT logging.** Confirm `kernel/rtt.c` output is visible through the new `stm32f407_rtt.cfg` OpenOCD session (`make view-rtt`-equivalent).
9. **Benchmarks.** Re-run `benchmark/bench_clock.c` and `benchmark/bench_yield_app.c` and record new baseline numbers — DWT cycle counts are clock-speed-relative and the F4 numbers are not comparable to the existing F1 baseline without renormalizing by clock ratio.

9. Summary Table

Area	File(s)	Change needed
Build flags	`Makefile`	Yes — new `CPU`, new `BOARD` branch
FPU context switch	`arch/arm_cm/port.c`	No, unless hardware float is adopted later (§3)
MPU setup	`arch/arm_cm/port.c`	No — PMSAv7 is shared between M3/M4
Vector table / Reset_Handler	`arch/arm_cm/startup.c`	No — core exception set is identical
Clock bring-up	`bsp/stm32f407/clock.c` (new)	Yes — full rewrite, different PLL/voltage/wait-state model
SysTick	`bsp/stm32f407/systick.c` (new)	No — copy verbatim, core peripheral
Memory map	`linker/stm32f407.ld` (new)	Yes — new sizes, re-derive power-of-two RAM split
Renode platform	`scripts/renode/stm32f407.*` (new)	Yes — new platform description
OpenOCD RTT config	`scripts/openocd/stm32f407_rtt.cfg` (new)	Yes — swap target config file
Priority inheritance / scheduler	`kernel/*`	No — architecture-independent